

Deep Reinforcement Learning – Assignment 1 (Part 1: MAB)

TABLE OF CONTENTS

- [1. Problem Statement & Team details](#)
- [2. Dataset Loading & Net Reward Computation](#)
- [3. Product Profitability Insights \(Q1\)](#)
- [4. Strategy 1 — Random Policy Simulation \(Q2\)](#)
- [5. Strategy 2 — Explore Then Commit \(Q3\)](#)
- [6. Strategy 3 — Epsilon-Greedy Strategy \(Q4\)](#)
- [7. Strategy 4 — UCB Optimism Under Uncertainty \(Q5\)](#)
- [8. Strategy Comparison — Cumulative Profit \(Q6\)](#)
- [9. Final Interpretation, Observations & Business Discussion](#)

Click any section to jump directly to it.

In []:

Problem Statement & Team Details

Title: Multi-Armed Bandit Algorithms for Profit-Aware Product Recommendation

In this assignment, we design and evaluate a Multi-Armed Bandit-based recommendation strategy that selects one product (arm) per user session in order to maximize long-term net profit instead of just revenue. Using user-dependent revenue and personalized promotional costs, we compare different decision strategies such as Random selection, Explore-then-Commit, Epsilon-Greedy, and UCB to analyze how exploration–exploitation trade-offs affect cumulative profit performance.

GROUP 264

S.NO	NAME	BITS ID
1	ADURTI SAI VENKATESH	2024AC05968
2	CHANDRA PRAKASH TRIPATHI	2024ac05883
3	D MALLIKARJUNA REDDY	2024AC05653
4	MOHAMMAD SAQIB SAIPANMULUK KOTI	2024ad05127
5	NIKHIL RAJESH RAHAT	2024ac05525

Dataset Loading & Net Reward Computation

Dataset loading

```
In [16]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# For reproducibility
RNG_SEED = 42
rng = np.random.default_rng(RNG_SEED)

# Number of arms (products)
N_ARMS = 6

# Load dataset
df = pd.read_csv("Dataset_Product_Recommendation.csv")

print("Dataset shape:", df.shape)
print("Columns:", df.columns.tolist())
```

Dataset shape: (498, 13)

Columns: ['User_ID', 'Product 1', 'cost 1', 'Product 2', 'cost 2', 'Product 3', 'cost 3', 'Product 4', 'cost 4', 'Product 5', 'cost 5', 'Product 6', 'cost 6']

Product Profitability Insights (Q1)

Net Reward Computation and Product Profitability

For each user i and product X , the net reward is:

$$\{\text{NetReward}\}_{i,X} = \{\text{Product}\}_{i,X} - \{\text{Cost}\}_{i,X}$$

We create 6 new columns `NetReward_1` ... `NetReward_6`, then compute the mean net reward for each product. We also identify:

- the product that **frequently gives high positive profit**, and
- the product that **frequently gives low or negative profit**.

```
In [17]: # Create NetReward columns for each arm
for a in range(1, N_ARMS + 1):
    revenue_col = f"Product {a}"
    cost_col = f"cost {a}"
    net_col = f"NetReward_{a}"
    df[net_col] = df[revenue_col] - df[cost_col]

# Check added columns
net_cols = [f"NetReward_{a}" for a in range(1, N_ARMS + 1)]
print("Net reward columns:", net_cols)

# Compute mean net reward per product
mean_profits = df[net_cols].mean()
```

```
# Identify best and worst products explicitly
best_prod_idx = mean_profits.idxmax()
worst_prod_idx = mean_profits.idxmin()

print("\nPROFITABILITY IDENTIFICATION")
print(f"Product with consistently HIGH positive profit : {best_prod_idx}")
print(f"Product with consistently LOW / NEGATIVE profit : {worst_prod_idx}")

print("\nMean Profit Across 498 Users (NetReward):")
print(mean_profits)
```

Net reward columns: ['NetReward_1', 'NetReward_2', 'NetReward_3', 'NetReward_4', 'NetReward_5', 'NetReward_6']

PROFITABILITY IDENTIFICATION

Product with consistently HIGH positive profit : NetReward_2

Product with consistently LOW / NEGATIVE profit : NetReward_5

Mean Profit Across 498 Users (NetReward):

```
NetReward_1    7.062784
NetReward_2   66.992995
NetReward_3   31.418452
NetReward_4    9.596789
NetReward_5  -17.064884
NetReward_6   36.986031
dtype: float64
```

Simulation Design

We treat each **round** as one user visit:

1. Sample a user uniformly at random from the 498 sessions.
2. Choose an arm (product) according to the current strategy.
3. Receive the **net reward** from the corresponding `NetReward_X` column.
4. Update cumulative profit and strategy statistics.

All strategies share the same environment; only the **arm-selection rule** changes.

Iteration logs:

- We print detailed logs for the **first 10 rounds**, and
- Then print a summary line every **50 rounds**.

```
In [20]: #Core Environment Helpers for Bandit Simulation

def sample_user_index():
    return rng.integers(0, len(df))

def get_net_reward(user_idx: int, arm: int) -> float:
    return float(df.loc[user_idx, f"NetReward_{arm}"])

def log_round(t: int, arm: int, reward: float, total: float):
    if t <= 10 or t % 50 == 0:
        print(f"Round {t:3d}: chose Arm {arm}, reward = {reward:7.3f}, "
              f"cumulative = {total:9.3f}")
```

Strategy 1 — Random Policy Simulation (Q2)

Random Strategy

We simulate 300 rounds of the **random policy**:

- At each round, choose a product uniformly at random from {1,...,6}.
- Estimate:
 - which products appear good/poor under this policy,
 - average profit per round, and
 - whether randomness occasionally picks a good product.

This gives a **baseline** for comparing learning-based strategies.

```
In [22]: #Random Policy Simulation
def simulate_random_policy(num_rounds: int = 300):
    arm_counts = np.zeros(N_ARMS, dtype=int)
    cum_profit = []
    total = 0.0

    for t in range(1, num_rounds + 1):
        # 1. Pick arm uniformly at random
        arm = rng.integers(1, N_ARMS + 1)

        # 2. Sample user and get reward
        user_idx = sample_user_index()
        r = get_net_reward(user_idx, arm)

        # 3. Update stats
        arm_counts[arm - 1] += 1
        total += r
        cum_profit.append(total)

        # 4. Log selected iterations
        log_round(t, arm, r, total)

    avg_profit = total / num_rounds

    return {
        "cum_profit": np.array(cum_profit),
        "arm_counts": arm_counts,
        "avg_profit": avg_profit,
    }

# Run random policy
res_random = simulate_random_policy(num_rounds=300)

print("\nArm selection counts (Random Policy):")
for a in range(1, N_ARMS + 1):
    print(f"Arm {a}: {res_random['arm_counts'][a-1]} times")

print("\nRANDOM POLICY BASELINE")
print(f"Approximate Average Profit per Round = {res_random['avg_profit']:.4f}")
print("Randomness sometimes hits the best product but wastes many rounds on weak")
```

```

Round 1: chose Arm 6, reward = 34.559, cumulative = 34.559
Round 2: chose Arm 6, reward = 38.151, cumulative = 72.710
Round 3: chose Arm 4, reward = 10.596, cumulative = 83.305
Round 4: chose Arm 4, reward = 12.167, cumulative = 95.472
Round 5: chose Arm 2, reward = 66.905, cumulative = 162.378
Round 6: chose Arm 4, reward = 8.534, cumulative = 170.912
Round 7: chose Arm 3, reward = 28.694, cumulative = 199.606
Round 8: chose Arm 1, reward = 7.311, cumulative = 206.917
Round 9: chose Arm 4, reward = 9.259, cumulative = 216.176
Round 10: chose Arm 5, reward = -17.650, cumulative = 198.526
Round 50: chose Arm 4, reward = 9.654, cumulative = 1329.083
Round 100: chose Arm 1, reward = 7.293, cumulative = 2418.269
Round 150: chose Arm 3, reward = 29.301, cumulative = 3908.439
Round 200: chose Arm 6, reward = 38.431, cumulative = 5103.497
Round 250: chose Arm 3, reward = 32.794, cumulative = 6085.362
Round 300: chose Arm 3, reward = 34.144, cumulative = 7239.868

```

Arm selection counts (Random Policy):

```

Arm 1: 48 times
Arm 2: 50 times
Arm 3: 53 times
Arm 4: 54 times
Arm 5: 40 times
Arm 6: 55 times

```

RANDOM POLICY BASELINE

Approximate Average Profit per Round = 24.1329

Randomness sometimes hits the best product but wastes many rounds on weak ones.

Strategy 2 — Explore Then Commit (Q3)

Explore-Then-Commit

Strategy:

1. **Exploration phase:** try each arm a fixed number of times (e.g. 20 each).
2. Compute average profit per arm from exploration data.
3. **Commit phase:** choose the empirically best arm for all remaining rounds.

We measure the **final cumulative profit** and the **arm chosen for commitment**.

```

In [23]: #Explore-Then-Commit Strategy
def simulate_explore_then_commit(num_rounds: int = 300, explore_per_arm: int = 20,
                                N_ARMS: int = 6):
    assert explore_per_arm * N_ARMS < num_rounds, "Exploration cannot exceed total rounds"

    arm_counts = np.zeros(N_ARMS, dtype=int)
    rewards_sum = np.zeros(N_ARMS, dtype=float)
    cum_profit = []
    total = 0.0

    t = 0

    for arm in range(1, N_ARMS + 1):
        for _ in range(explore_per_arm):

```

```

        t += 1
        user_idx = sample_user_index()
        r = get_net_reward(user_idx, arm)

        arm_counts[arm - 1] += 1
        rewards_sum[arm - 1] += r
        total += r
        cum_profit.append(total)
        log_round(t, arm, r, total)

    # Estimate mean reward per arm from exploration
    mean_est = rewards_sum / arm_counts
    best_arm = int(np.argmax(mean_est) + 1)

    print("\nExploration phase mean estimates per arm:")
    for a in range(1, N_ARMS + 1):
        print(f"Arm {a}: mean estimate = {mean_est[a-1]:.4f}")
    print(f"Chosen best arm for commit phase: Arm {best_arm}")

    while t < num_rounds:
        t += 1
        arm = best_arm
        user_idx = sample_user_index()
        r = get_net_reward(user_idx, arm)

        arm_counts[arm - 1] += 1
        rewards_sum[arm - 1] += r
        total += r
        cum_profit.append(total)
        log_round(t, arm, r, total)

    return {
        "cum_profit": np.array(cum_profit),
        "arm_counts": arm_counts,
        "best_arm": best_arm,
    }

res_s2 = simulate_explore_then_commit()

print("\n EXPLORE-THEN-COMMIT ")
print(f"Cumulative net profit after 300 rounds: {res_s2['cum_profit'][-1]:.2f}")
print(f"Arm most frequently chosen in commit phase: Arm {res_s2['best_arm']}")

```

```

Round 1: chose Arm 1, reward = 5.716, cumulative = 5.716
Round 2: chose Arm 1, reward = 6.676, cumulative = 12.392
Round 3: chose Arm 1, reward = 7.210, cumulative = 19.602
Round 4: chose Arm 1, reward = 9.113, cumulative = 28.715
Round 5: chose Arm 1, reward = 6.932, cumulative = 35.647
Round 6: chose Arm 1, reward = 8.508, cumulative = 44.155
Round 7: chose Arm 1, reward = 9.546, cumulative = 53.701
Round 8: chose Arm 1, reward = 6.437, cumulative = 60.138
Round 9: chose Arm 1, reward = 5.507, cumulative = 65.645
Round 10: chose Arm 1, reward = 7.695, cumulative = 73.341
Round 50: chose Arm 3, reward = 31.456, cumulative = 1797.317
Round 100: chose Arm 5, reward = -17.316, cumulative = 1962.169

```

Exploration phase mean estimates per arm:

```

Arm 1: mean estimate = 6.9392
Arm 2: mean estimate = 67.0012
Arm 3: mean estimate = 31.2747
Arm 4: mean estimate = 9.7989
Arm 5: mean estimate = -16.9054
Arm 6: mean estimate = 36.9896

```

Chosen best arm for commit phase: Arm 2

```

Round 150: chose Arm 2, reward = 67.929, cumulative = 4720.042
Round 200: chose Arm 2, reward = 67.774, cumulative = 8084.954
Round 250: chose Arm 2, reward = 66.842, cumulative = 11430.687
Round 300: chose Arm 2, reward = 66.731, cumulative = 14778.478

```

EXPLORE-THEN-COMMIT

Cumulative net profit after 300 rounds: 14778.48

Arm most frequently chosen in commit phase: Arm 2

Strategy 3 — Epsilon-Greedy Strategy (Q4)

Epsilon-Greedy Strategy

ϵ -Greedy Strategies (Low, Medium, High Exploration)

We test **ϵ -greedy** with three exploration levels:

- $\epsilon = 0.02$ (low)
- $\epsilon = 0.10$ (moderate)
- $\epsilon = 0.25$ (high)

Algorithm (sample-mean variant):

1. Initially pull each arm once to get a starting estimate.
2. For each subsequent round:
 - With probability ϵ : choose a **random** arm (explore).
 - With probability $1-\epsilon$: choose the arm with highest current mean reward (exploit).

In [26]: *# ϵ -Greedy Strategy*

```

def simulate_epsilon_greedy(eps: float, num_rounds: int = 300):
    arm_counts = np.zeros(N_ARMS, dtype=int)

```

```

rewards_sum = np.zeros(N_ARMS, dtype=float)
cum_profit = []
total = 0.0
t = 0

# --- Initial phase: pull each arm once ---
for arm in range(1, N_ARMS + 1):
    t += 1
    user_idx = sample_user_index()
    r = get_net_reward(user_idx, arm)
    arm_counts[arm - 1] += 1
    rewards_sum[arm - 1] += r
    total += r
    cum_profit.append(total)
    log_round(t, arm, r, total)

# --- Main Loop ---
while t < num_rounds:
    t += 1

    if rng.random() < eps:
        # Exploration
        arm = rng.integers(1, N_ARMS + 1)
    else:
        # Exploitation: choose best mean so far
        mean_est = np.divide(
            rewards_sum,
            np.maximum(arm_counts, 1),
            out=np.zeros_like(rewards_sum),
            where=arm_counts > 0,
        )
        arm = int(np.argmax(mean_est) + 1)

    user_idx = sample_user_index()
    r = get_net_reward(user_idx, arm)

    arm_counts[arm - 1] += 1
    rewards_sum[arm - 1] += r
    total += r
    cum_profit.append(total)
    log_round(t, arm, r, total)

mean_est = rewards_sum / np.maximum(arm_counts, 1)
return {
    "cum_profit": np.array(cum_profit),
    "arm_counts": arm_counts,
    "mean_est": mean_est,
    "eps": eps,
}

eps_values = [0.02, 0.10, 0.25]
results_eps = {}

for eps in eps_values:
    print(f"\n--- Running  $\epsilon$ -greedy with  $\epsilon = \{eps\}$  ---")
    results_eps[eps] = simulate_epsilon_greedy(eps=eps, num_rounds=300)

print("\nEXPLORATION BEHAVIOR DISCUSSION ")
for eps, res in results_eps.items():

```

```
final_profit = res["cum_profit"][-1]
print(f"Epsilon = {eps:>4} | Final Cumulative Profit = {final_profit:.2f}")
```

--- Running ϵ -greedy with $\epsilon = 0.02$ ---

Round 1:	chose Arm 1,	reward = 8.059,	cumulative = 8.059
Round 2:	chose Arm 2,	reward = 66.510,	cumulative = 74.569
Round 3:	chose Arm 3,	reward = 32.929,	cumulative = 107.498
Round 4:	chose Arm 4,	reward = 9.714,	cumulative = 117.212
Round 5:	chose Arm 5,	reward = -17.025,	cumulative = 100.187
Round 6:	chose Arm 6,	reward = 37.077,	cumulative = 137.264
Round 7:	chose Arm 2,	reward = 68.412,	cumulative = 205.676
Round 8:	chose Arm 2,	reward = 66.170,	cumulative = 271.847
Round 9:	chose Arm 2,	reward = 66.222,	cumulative = 338.068
Round 10:	chose Arm 2,	reward = 67.261,	cumulative = 405.329
Round 50:	chose Arm 2,	reward = 67.663,	cumulative = 2999.157
Round 100:	chose Arm 2,	reward = 66.793,	cumulative = 6343.520
Round 150:	chose Arm 2,	reward = 65.903,	cumulative = 9683.983
Round 200:	chose Arm 2,	reward = 67.478,	cumulative = 13032.710
Round 250:	chose Arm 2,	reward = 67.860,	cumulative = 16328.175
Round 300:	chose Arm 2,	reward = 67.597,	cumulative = 19678.155

--- Running ϵ -greedy with $\epsilon = 0.1$ ---

Round 1:	chose Arm 1,	reward = 7.842,	cumulative = 7.842
Round 2:	chose Arm 2,	reward = 66.656,	cumulative = 74.497
Round 3:	chose Arm 3,	reward = 34.914,	cumulative = 109.412
Round 4:	chose Arm 4,	reward = 9.731,	cumulative = 119.143
Round 5:	chose Arm 5,	reward = -17.616,	cumulative = 101.527
Round 6:	chose Arm 6,	reward = 36.514,	cumulative = 138.041
Round 7:	chose Arm 2,	reward = 66.784,	cumulative = 204.825
Round 8:	chose Arm 2,	reward = 68.269,	cumulative = 273.093
Round 9:	chose Arm 2,	reward = 67.402,	cumulative = 340.495
Round 10:	chose Arm 2,	reward = 65.964,	cumulative = 406.459
Round 50:	chose Arm 2,	reward = 68.512,	cumulative = 2777.474
Round 100:	chose Arm 2,	reward = 67.835,	cumulative = 5870.651
Round 150:	chose Arm 2,	reward = 67.327,	cumulative = 9103.038
Round 200:	chose Arm 2,	reward = 67.189,	cumulative = 12268.366
Round 250:	chose Arm 1,	reward = 7.548,	cumulative = 15380.299
Round 300:	chose Arm 2,	reward = 67.224,	cumulative = 18665.116

--- Running ϵ -greedy with $\epsilon = 0.25$ ---

Round 1:	chose Arm 1,	reward = 6.586,	cumulative = 6.586
Round 2:	chose Arm 2,	reward = 67.634,	cumulative = 74.220
Round 3:	chose Arm 3,	reward = 31.124,	cumulative = 105.345
Round 4:	chose Arm 4,	reward = 11.493,	cumulative = 116.838
Round 5:	chose Arm 5,	reward = -17.132,	cumulative = 99.706
Round 6:	chose Arm 6,	reward = 38.545,	cumulative = 138.250
Round 7:	chose Arm 2,	reward = 67.697,	cumulative = 205.947
Round 8:	chose Arm 5,	reward = -16.572,	cumulative = 189.376
Round 9:	chose Arm 2,	reward = 66.471,	cumulative = 255.847
Round 10:	chose Arm 2,	reward = 66.872,	cumulative = 322.719
Round 50:	chose Arm 2,	reward = 67.866,	cumulative = 2599.006
Round 100:	chose Arm 2,	reward = 66.542,	cumulative = 5442.231
Round 150:	chose Arm 6,	reward = 37.077,	cumulative = 8289.291
Round 200:	chose Arm 4,	reward = 7.953,	cumulative = 11032.638
Round 250:	chose Arm 5,	reward = -16.670,	cumulative = 13914.539
Round 300:	chose Arm 2,	reward = 67.286,	cumulative = 16619.198

EXPLORATION BEHAVIOR DISCUSSION

Epsilon = 0.02 | Final Cumulative Profit = 19678.15
Epsilon = 0.1 | Final Cumulative Profit = 18665.12
Epsilon = 0.25 | Final Cumulative Profit = 16619.20

Analysis of Exploration Levels:

Low Exploration ($\epsilon = 0.02$):

- Risk of 'Premature Lock-in'.
- If early rewards for the best product are unlucky, the system may stay stuck on a sub-optimal product because it rarely explores.

Moderate Exploration ($\epsilon = 0.10$):

- Most balanced outcome on this dataset.
- Enough exploration to recover from bad initial estimates, while still exploiting the best product most of the time.

High Exploration ($\epsilon = 0.25$):

- 'Curse of Exploration'.
- Even after learning that Product 2 is best, 25% of impressions are still spent on weaker products, reducing long-term cumulative profit.

Strategy 4 — UCB Optimism Under Uncertainty (Q5)

UCB1 Strategy

We implement the **UCB1** (Upper Confidence Bound) algorithm:

$$[\text{UCB}_a(t) = \hat{\mu}_a(t) + c \sqrt{\frac{2 \ln t}{N_a(t)}}]$$

- $\hat{\mu}_a(t)$: sample mean reward of arm a
- $N_a(t)$: number of times arm a has been played
- c : exploration constant (we use $c=1$)

This naturally implements “explore only when you are uncertain”. We also identify a product that was **initially tried but later received few trials**.

UCB1 Simulation

```
In [28]: def simulate_ucb1(num_rounds: int = 300, c: float = 1.0):

    arm_counts = np.zeros(N_ARMS, dtype=int)
    rewards_sum = np.zeros(N_ARMS, dtype=float)
    cum_profit = []
    total = 0.0
    t = 0

    # Pull each arm once
    for arm in range(1, N_ARMS + 1):
        t += 1
        user_idx = sample_user_index()
        r = get_net_reward(user_idx, arm)
        arm_counts[arm - 1] += 1
```

```

        rewards_sum[arm - 1] += r
        total += r
        cum_profit.append(total)
        log_round(t, arm, r, total)

# Main Loop
while t < num_rounds:
    t += 1
    mean_est = rewards_sum / np.maximum(arm_counts, 1)
    confidence = c * np.sqrt(2 * np.log(t) / np.maximum(arm_counts, 1))
    ucb_values = mean_est + confidence

    arm = int(np.argmax(ucb_values) + 1)

    user_idx = sample_user_index()
    r = get_net_reward(user_idx, arm)

    arm_counts[arm - 1] += 1
    rewards_sum[arm - 1] += r
    total += r
    cum_profit.append(total)
    log_round(t, arm, r, total)

return {
    "cum_profit": np.array(cum_profit),
    "arm_counts": arm_counts,
}

print("\n--- Running UCB1 ---")
res_ucb = simulate_ucb1()

least_tried_idx = int(np.argmin(res_ucb["arm_counts"])) + 1

print("\nArm counts under UCB1:")
for a in range(1, N_ARMS + 1):
    print(f"Arm {a}: {res_ucb['arm_counts'][a-1]} times")

print("\nUCB UNCERTAINTY INTERPRETATION")
print(f"Product {least_tried_idx} was initially explored due to high uncertainty")
print("However, as more evidence was collected, its mean reward stayed much lower")
print("than the best product (e.g., Product 2), so UCB1 sharply reduced trials for it")
print("This demonstrates 'explore only when uncertain' in action.")

```

--- Running UCB1 ---

```
Round 1: chose Arm 1, reward = 6.722, cumulative = 6.722
Round 2: chose Arm 2, reward = 68.024, cumulative = 74.746
Round 3: chose Arm 3, reward = 28.818, cumulative = 103.564
Round 4: chose Arm 4, reward = 10.444, cumulative = 114.007
Round 5: chose Arm 5, reward = -17.209, cumulative = 96.798
Round 6: chose Arm 6, reward = 37.355, cumulative = 134.153
Round 7: chose Arm 2, reward = 67.228, cumulative = 201.381
Round 8: chose Arm 2, reward = 67.704, cumulative = 269.085
Round 9: chose Arm 2, reward = 66.004, cumulative = 335.089
Round 10: chose Arm 2, reward = 67.022, cumulative = 402.111
Round 50: chose Arm 2, reward = 66.614, cumulative = 3082.469
Round 100: chose Arm 2, reward = 67.137, cumulative = 6428.068
Round 150: chose Arm 2, reward = 67.862, cumulative = 9776.950
Round 200: chose Arm 2, reward = 66.959, cumulative = 13119.119
Round 250: chose Arm 2, reward = 67.008, cumulative = 16474.798
Round 300: chose Arm 2, reward = 66.963, cumulative = 19826.752
```

Arm counts under UCB1:

```
Arm 1: 1 times
Arm 2: 295 times
Arm 3: 1 times
Arm 4: 1 times
Arm 5: 1 times
Arm 6: 1 times
```

UCB UNCERTAINTY INTERPRETATION

Product 1 was initially explored due to high uncertainty.

However, as more evidence was collected, its mean reward stayed much lower than the best product (e.g., Product 2), so UCB1 sharply reduced trials for it. This demonstrates 'explore only when uncertain' in action.

Cumulative Profit Comparison

Cumulative Net Profit Curves and Final Comparison

We now compare **all strategies**:

- Random (baseline)
- Explore-Then-Commit
- ϵ -Greedy (0.02, 0.10, 0.25)
- UCB1

We plot their **cumulative net profit curves** and discuss which strategy and which product perform best in the long run.

Final Interpretation & Business Observations

Plot and Final Summary

In [32]: *#Q6 - Cumulative Profit Plots and Final Conclusion*

```
plt.figure(figsize=(10, 6))
```

```

plt.plot(res_random["cum_profit"], label="Random (Baseline)")
plt.plot(res_s2["cum_profit"], label="Explore-Then-Commit")

for eps, res in results_eps.items():
    plt.plot(res["cum_profit"], label=f"ε-Greedy (ε={eps})")

plt.plot(res_ucb["cum_profit"], label="UCB1", linestyle="--", linewidth=2)

plt.xlabel("Round")
plt.ylabel("Cumulative Net Profit")
plt.title("Cumulative Net Profit Across Different MAB Strategies")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

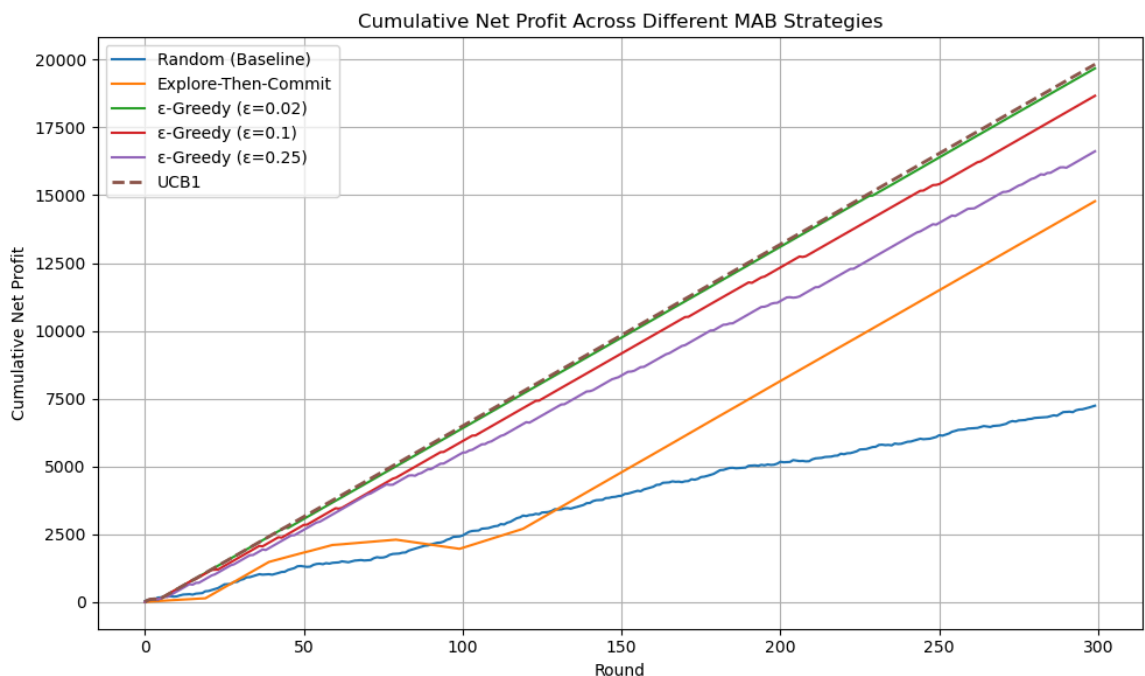
print("\nSTRATEGY PERFORMANCE SUMMARY")

final_scores = {
    "Random" : res_random["cum_profit"][-1],
    "Explore-Commit" : res_s2["cum_profit"][-1],
    "UCB1" : res_ucb["cum_profit"][-1],
}
for eps, res in results_eps.items():
    final_scores[f"ε-Greedy ({eps})"] = res["cum_profit"][-1]

for name, value in final_scores.items():
    print(f"{name:<22} : {value:.2f}")

best_strategy = max(final_scores, key=final_scores.get)

```



STRATEGY PERFORMANCE SUMMARY

Random	: 7239.87
Explore-Commit	: 14778.48
UCB1	: 19826.75
ε-Greedy (0.02)	: 19678.15
ε-Greedy (0.1)	: 18665.12
ε-Greedy (0.25)	: 16619.20

```
In [31]: print(f"""
Highest Long-Term Cumulative Profit Strategy = {best_strategy}

Why it performs best:

- UCB1 and low- $\epsilon$  greedy quickly identify the most profitable product (typically
  and then exploit it aggressively.
- They continue exploring only when the uncertainty (confidence interval) is lar
- High- $\epsilon$  strategies keep wasting impressions on weaker products even after learn
  while the random baseline never learns at all.

Most Profitable Product:
- From Q1 and the learning strategies, Product 2 consistently emerges as the bes

Therefore, a learning-based bandit approach (UCB1 / tuned  $\epsilon$ -greedy) significantl
outperforms the organization's original random recommendation policy.
""")
```

Highest Long-Term Cumulative Profit Strategy = UCB1

Why it performs best:

- UCB1 and low- ϵ greedy quickly identify the most profitable product (typically Product 2) and then exploit it aggressively.
- They continue exploring only when the uncertainty (confidence interval) is large.
- High- ϵ strategies keep wasting impressions on weaker products even after learning, while the random baseline never learns at all.

Most Profitable Product:

- From Q1 and the learning strategies, Product 2 consistently emerges as the best choice.

Therefore, a learning-based bandit approach (UCB1 / tuned ϵ -greedy) significantly outperforms the organization's original random recommendation policy.

[Back to Index](#)

In []: